NOTE ASP.NET Core comes with such a filter out of the box: `AuthorizeFilter`. I discuss this filter in chapter 22, and you'll be seeing a lot more of it in chapter 24.

As I described in the previous section, you normally create filters as attributes, and for good reason: it makes it easy for you to apply them to MVC controllers, actions, and Razor Pages. In this section you'll see how to apply `LogResourceFilter` from listing 21.1 to an action, a controller, a Razor Page, and globally. The level at which the filter applies is called its *scope*.

DEFINITION The *scope* of a filter refers to how many different actions it applies to. A filter can be scoped to the action method, to the controller, to a Razor Page, or globally.

You'll start at the most specific scope: applying filters to a single action. The following listing shows an example of an MVC controller that has two action methods, one with `LogResourceFilter` and one without.

Listing 21.3 Applying filters to an action method

```
public class RecipeController : ControllerBase
{
    [LogResourceFilter]                ❶
    public IActionResult Index()       ❶
    {                                  ❶
        return Ok();                   ❶
    }                                  ❶
    public IActionResult View()        ❷
    {                                  ❷
        return OK();                   ❷
    }                                  ❷
}
```

❶ LogResourceFilter runs as part of the pipeline when executing this action.

❷ This action method has no filters at the action level.